WORLD 2 / TYPESCRIPT

TypeScript | Signal Harbor

Design reliable contracts for JavaScript programs

Signal Harbor routes cargo through a network of manifests, message queues, and partner docks. You are rebuilding the dispatch desk. Each level introduces one way to keep a program's promises visible as data moves between people, functions, and systems.

Begin with the Dockhand path if you are new to TypeScript. Work through the code in order. For each listing, predict a result, change one input, and check whether the compiler and the running program behave as you expected. The lessons use familiar JavaScript expressions and build up their type contracts gradually.

The Contract Engineer path develops APIs that express relationships between values. The Boundary Operator path emphasizes validation, failure, and integration. Check each standalone listing with tsc --strict --target ES2022 --noEmit. Node.js can run these listings by stripping the types, so run the compiler separately to check them. Validate external data at runtime.

Start with the beginner route if these ideas are new to you. In the game, choose a hero to save your checkpoint answers, code drafts, and quest log. The attached field kit contains the worked examples, starter files, and a quest journal.

Explore the world map · Begin Level 01

World map

Dockhand

Begin with everyday values, then build a small dispatch program you can understand from input to output.

Contract Engineer

Make call sites carry the right data by designing types around the relationships between values.

Boundary Operator

Track the work that enters and leaves a system, including invalid input, asynchronous failure, and repeat delivery.

The Arrival Docks

01 / The Cargo Counter
02 / The Manifest Shape
03 / The Two Signal Flags
04 / Callbacks at the Sorting Belt

The Signal Office

05 / The Untrusted Manifest
06 / Waiting for the Tide Report
07 / The Dispatch Desk

The Contract Foundry

08 / Keys That Carry Their Values
09 / The Registry Blueprint
10 / States That Refuse to Drift
11 / The Typed Message Board

The Outer Harbor

12 / Dispatching Several Calls
13 / Stamping an Identifier
14 / Restore the Harbor Dispatch

LEVEL 01 / FIELD MANUAL

The Cargo Counter

Use inference for local values and annotations for the inputs and outputs of a small function.

TypeScript checks JavaScript programs before they run. Start with three common kinds of values: numbers for arithmetic, strings for text, and booleans for decisions. A declaration such as `let load = 3` lets the checker infer `number`, so later assigning a string to `load` produces an error. `const` prevents reassignment and can preserve the exact literal type for a primitive value.

A function annotation describes the contract callers should follow. `mass: number` gives the function a numeric input, and `: string` after the parameter list commits it to returning text. Try returning `mass` directly. The return annotation catches the mistake inside the function, where the change is easiest to understand.

JavaScript still allows a `number` to be `NaN`, infinite, or negative. A type can establish that arithmetic is available, while a runtime condition establishes a domain rule. The example checks both inputs before multiplying and rejects an infinite result, which can occur even when both inputs are finite.

Worked example / Level 01

Listing 1.1 TypeScript example

```
function cargoLabel(count: number, unitMass: number): string {
  if (!Number.isSafeInteger(count) || count < 0) {
    throw new RangeError('Invalid cargo count');
  }
  if (!Number.isFinite(unitMass) || unitMass < 0) {
    throw new RangeError('Invalid unit mass');
  }
  const mass = count * unitMass;
  if (!Number.isFinite(mass)) throw new RangeError('Mass overflow');
  return `${count} units, ${mass} kg`;
}

const label = cargoLabel(3, 2.5);
if (label !== '3 units, 7.5 kg') throw new Error(label);
```

Remember. Type annotations describe permitted kinds of values. Runtime checks enforce business rules within those kinds.

Try the encounter

LEVEL 01 / FORGE

The encounter

Checkpoint

Which value is allowed by the annotation `count: number` but should fail this cargo rule?

A. The string 'three'

B. The boolean true

C. The number 2.5

D. The value undefined

Code challenge

Implement palletLabel. A pallet carries ten crates. Return the number of pallets needed for a nonnegative safe integer crate count. Use the exact format `Pallets: 2` for 11 crates and throw RangeError for an invalid count.

Level 1 starter

```
function palletLabel(crateCount: number): string {
  // Check the count, then round the pallet requirement up.
  throw new Error('Implement palletLabel');
}
```

Read the debrief · Back to the field manual

LEVEL 01 / EXPLAIN

Debrief and side quest

Checkpoint. C. The value 2.5 has type number, but the count must be an integer. The function needs the integer check even in a type-checked program.

Hint one

Use `Number.isSafeInteger(crateCount)` before performing arithmetic.

Hint two

`Math.ceil(crateCount / 10)` gives the number of pallets. Check the empty case with zero crates.

Compare your solution

A guard rejects negative, fractional, and non-finite counts. Division followed by rounding up handles exact multiples and partial pallets with one expression. Check 0, 10, and 11 crates to catch boundary errors.

Side quest

Replace the fixed capacity with a parameter. Decide which capacities are valid and test an invalid capacity before checking the result.

Continue to Level 02 · World map

LEVEL 02 / FIELD MANUAL

The Manifest Shape

Describe a record and let a function read a collection without granting it permission to mutate the input.

A cargo record has named fields. A type alias gives that shape a reusable name. Functions can state which fields they need, and TypeScript checks whether the supplied value has those fields with compatible types. An object held in a variable may carry extra fields and still be passed to a function expecting the smaller shape.

`readonly Cargo[]` makes the array read-only through that parameter. The function can iterate and inspect its entries. It cannot push, sort in place, or assign a replacement element through that reference. Put `readonly` on the record fields as well if the function should be prevented from assigning through them.

Read-only types apply to the reference the checker sees. JavaScript may still hold another alias with write access. If a report must remain stable after the caller edits its own objects, construct a snapshot of the required data and decide how deeply it must be copied.

Worked example / Level 02

Listing 2.1 TypeScript example

```
type Cargo = {
  readonly id: string;
  readonly weightKg: number;
};

function totalWeight(items: readonly Cargo[]): number {
  let total = 0;
  for (const item of items) total += item.weightKg;
  return total;
}

const manifest = [
  { id: 'A-1', weightKg: 12, owner: 'Ferry Office' },
  { id: 'A-2', weightKg: 8, owner: 'Warehouse' },
];
if (totalWeight(manifest) !== 20) throw new Error('Bad total');
```

Remember. Describe the fields a caller must supply and the write access a callee should receive.

Try the encounter

LEVEL 02 / FORGE

The encounter

Checkpoint

What does `readonly Cargo[]` prevent inside `totalWeight`?

A. Passing an array with extra fields on its records

B. Changing the array through the parameter

C. Another alias changing the original array

D. A weight being negative at runtime

Code challenge

Return the IDs of all cargo weighing at least 10 kg, in the same order as the input. Create a new array and leave the manifest alone.

Level 2 starter

```
type Cargo = { readonly id: string; readonly weightKg: number };

function heavyIds(items: readonly Cargo[]): string[] {
  throw new Error('Implement heavyIds');
}
```

Read the debrief · Back to the field manual

LEVEL 02 / EXPLAIN

Debrief and side quest

Checkpoint. B. The read-only parameter restricts writes through that parameter. Record field mutability and other aliases have their own rules.

Hint one

Try a loop that creates a fresh `string[]` before inspecting the first record.

Hint two

Add the ID when `weightKg >= 10`. Test a record weighing exactly 10 kg.

Compare your solution

Each selected ID is copied into a new array. The caller can add IDs to that output without changing the input array, and filtering preserves relative order.

Side quest

Design a grouped report whose results can be read but cannot be edited through the type returned to a consumer.

Continue to Level 03 · World map

LEVEL 03 / FIELD MANUAL

The Two Signal Flags

Narrow a union by checking a literal tag before reading fields that belong to one case.

An arrival either has a dock assignment or is waiting for a reason. Model both shapes with a union. Each shape carries a `kind` field with a different literal value. The compiler can follow a comparison with that field and make the corresponding shape available inside the branch.

A single object type with optional `dock` and `reason` fields would permit a waiting arrival with no reason. A tagged union makes the appropriate fields required for each case. A structurally compatible value can still carry extra properties. Check the tag before using the fields required for that branch.

When testing an optional primitive, check for the missing state you mean. `if (dock)` treats zero as false, even if dock zero is a valid location. `dock !== undefined` makes the distinction explicit. For a tagged union, checking `kind` avoids mixing a status decision with the truthiness of a field.

Worked example / Level 03

Listing 3.1 TypeScript example

```
type Arrival =
  | { kind: 'assigned'; dock: number }
  | { kind: 'waiting'; reason: string };

function statusText(arrival: Arrival): string {
  if (arrival.kind === 'assigned') {
    return `Dock ${arrival.dock}`;
  }
  return `Waiting: ${arrival.reason}`;
}

const status = statusText({ kind: 'assigned', dock: 0 });
if (status !== 'Dock 0') throw new Error(status);
```

Remember. A discriminant lets the compiler track which fields exist after a runtime check.

Try the encounter

LEVEL 03 / FORGE

The encounter

Checkpoint

Which property can you access after `arrival.kind === 'waiting'` succeeds?

A. reason

B. dock

C. Both reason and dock

D. Neither reason nor dock

Code challenge

Implement canUnload. An assigned arrival can unload only if its dock is open. A waiting arrival cannot unload. Use the dock list as read-only input.

Level 3 starter

```
type Arrival =
  | { kind: 'assigned'; dock: number }
  | { kind: 'waiting'; reason: string };

function canUnload(
  arrival: Arrival, openDocks: readonly number[]
): boolean {
  throw new Error('Implement canUnload');
}
```

Read the debrief · Back to the field manual

LEVEL 03 / EXPLAIN

Debrief and side quest

Checkpoint. A. The literal tag narrows the union to the waiting case, where `reason` is required.

Hint one

Narrow the union using `arrival.kind` before you access `arrival.dock`.

Hint two

Use `openDocks.includes(arrival.dock)` inside the assigned branch, or combine both conditions with `&&`.

Compare your solution

The status test establishes which shape is present. The membership test establishes whether that dock is open. Both decisions matter, including when the assignment is to dock zero.

Side quest

Add a diverted case carrying a destination harbor. Give it a distinct message in statusText.

Continue to Level 04 · World map

LEVEL 04 / FIELD MANUAL

Callbacks at the Sorting Belt

Pass a typed function as an argument and preserve meaningful falsy values when applying a default.

Two dock reports can traverse the same manifest while counting different things. Pass a rule as a function. A parameter of type `(cargo: Cargo) => boolean` promises that the report supplies a cargo record and expects the rule to say whether it belongs in the count.

An arrow function written directly at the call site usually inherits its parameter type from the expected callback type. The checker can suggest valid fields even if the arrow parameter has no annotation. Read the parameter type and the return type together when deciding whether a callback is suitable.

`??` applies its right-hand value only when the left side is `null` or `undefined`. The example accepts an optional maximum, validates any supplied maximum as a nonnegative safe integer, and defaults to no limit. A maximum of zero means select nothing. `??` preserves that value.

Worked example / Level 04

Listing 4.1 TypeScript example

```
type Cargo = { weightKg: number };

function countMatching(
  items: readonly Cargo[],
  matches: (cargo: Cargo) => boolean,
  maximum?: number,
): number {
  if (maximum !== undefined &&
      (!Number.isSafeInteger(maximum) || maximum < 0)) {
    throw new RangeError('Invalid maximum');
  }
  const limit = maximum ?? Infinity;
  let count = 0;
  for (const item of items) {
    if (count >= limit) break;
    if (matches(item)) count++;
  }
  return count;
}

const cargo = [{ weightKg: 12 }, { weightKg: 3 }];
const count = countMatching(cargo, item => item.weightKg > 10, 0);
if (count !== 0) throw new Error('Limit of zero was lost');
```

Remember. Give reusable traversal a callback contract and make the default policy explicit.

Try the encounter

LEVEL 04 / FORGE

The encounter

Checkpoint

Why does the example use `maximum ?? Infinity`?

A. It rejects negative maxima

B. It converts strings to numbers

C. It throws when the argument is absent

D. It preserves an explicit zero maximum

Code challenge

Implement labelMatching. Call label once per weight that passes weight >= 0 and return the labels in input order. Negative weights and NaN fail this test.

Level 4 starter

```
function labelMatching(
  weights: readonly number[],
  label: (weight: number) => string,
): string[] {
  throw new Error('Implement labelMatching');
}
```

Read the debrief · Back to the field manual

LEVEL 04 / EXPLAIN

Debrief and side quest

Checkpoint. D. Zero is distinct from `undefined`, so `??` keeps the zero and `||` would replace it.

Hint one

Build a fresh array. Use the positive test `weight >= 0` to decide whether to append a label.

Hint two

Call `label(weight)` once per retained item. Zero is nonnegative and should reach the callback.

Compare your solution

The traversal owns the decision to include an item. The callback owns the presentation. A callback that records its inputs can show whether the traversal invokes it for the correct items and in the right order.

Side quest

Add a test callback that throws on its second call. Explain what the caller observes and whether the partial result ever escapes.

Continue to Level 05 · World map

LEVEL 05 / FIELD MANUAL

The Untrusted Manifest

Turn untrusted JSON into a checked record by combining unknown with small runtime checks.

A partner can send malformed JSON or valid JSON with the wrong shape. A TypeScript interface describes the shape your program expects. Annotating a value with that interface cannot verify bytes received from a file or the network. Give parsed input the type `unknown` so the checker requires evidence before you use its fields.

A JSON parse may throw before a value exists. After parsing, require a non-null, non-array object. The `in` checks let TypeScript read the named fields as unknown. `Object.hasOwn` requires the fields on the record itself. Then check value types and domain rules such as a nonempty trimmed ID or a finite, nonnegative weight.

Construct a new record from the fields you validated. That tells readers which values crossed the boundary, and gives your internal type a clean starting point. An `as Manifest` assertion would change what the checker believes while leaving a malformed value untouched.

Worked example / Level 05

Listing 5.1 TypeScript example

```
type Manifest = { id: string; weightKg: number };

function readManifest(raw: string): Manifest | undefined {
  let value: unknown;
  try {
    value = JSON.parse(raw);
  } catch {
    return undefined;
  }
  if (typeof value !== 'object' || value === null) return undefined;
  if (Array.isArray(value)) return undefined;
  if (!('id' in value) || !Object.hasOwn(value, 'id')) return undefined;
  if (!('weightKg' in value) || !Object.hasOwn(value, 'weightKg')) {
    return undefined;
  }
  const { id, weightKg } = value;
  if (typeof id !== 'string' || id.trim() === '') return undefined;
  if (typeof weightKg !== 'number') return undefined;
  if (!Number.isFinite(weightKg) || weightKg < 0) return undefined;
  return { id: id.trim(), weightKg };
}

if (!readManifest('{"id":"C-4","weightKg":20}')) {
  throw new Error('Valid manifest rejected');
}
```

Remember. Let runtime evidence earn the internal type at every untrusted boundary.

Try the encounter

LEVEL 05 / FORGE

The encounter

Checkpoint

What does `JSON.parse(raw) as Manifest` verify at runtime?

A. That the parsed object has an id

B. That each field has its declared type

C. JSON syntax only

D. That a weight is nonnegative

Code challenge

Decode a dock status from raw JSON. Require a non-array record with its own nonnegative safe integer dock and boolean open. Return undefined for a syntax or shape error.

Level 5 starter

```
type DockStatus = { dock: number; open: boolean };

function readDockStatus(raw: string): DockStatus | undefined {
  throw new Error('Implement readDockStatus');
}
```

Read the debrief · Back to the field manual

LEVEL 05 / EXPLAIN

Debrief and side quest

Checkpoint. C. `JSON.parse` checks syntax and may throw. The assertion is erased during compilation, so it does not inspect the result's shape.

Hint one

Assign `JSON.parse` to an `unknown` variable inside a try block. Handle null before checking properties.

Hint two

Narrow both fields with `in` and require own properties with `Object.hasOwn`. Check `open` with `typeof` so `false` is accepted.

Compare your solution

A valid object with `dock: 0` and `open: false` should pass. The strings `"0"` and `"false"` should fail. Testing those pairs proves the implementation checks types instead of relying on truthiness or coercion.

Side quest

Return an error code that distinguishes invalid JSON from a missing field and a field with the wrong type.

Continue to Level 06 · World map

LEVEL 06 / FIELD MANUAL

Waiting for the Tide Report

Await a promise, handle rejection, and keep a real value distinct from a failed request.

An asynchronous operation can produce a value later. `Promise<number>` describes the value on fulfillment. Inside an async function, `await` obtains that value or throws when the promise rejects. The surrounding function still returns a promise, so its caller needs to await it or otherwise observe its outcome.

Place the await inside `try` when you want that function to translate a rejection. If the operation resolves to zero, zero is data. A separate error case lets the caller tell whether the harbor is quiet or the status report could not be fetched.

Rejected promises can carry arbitrary values. An error created with `new Error` has a message. A thrown string or plain object requires a separate policy. The example keeps its public failure message stable, while a real service might log a narrowed error privately for diagnosis.

Worked example / Level 06

Listing 6.1 TypeScript example

```
type TideReport =
  | { ok: true; heightCm: number }
  | { ok: false; message: string };

async function reportTide(
  readHeight: () => Promise<number>,
): Promise<TideReport> {
  try {
    const heightCm = await readHeight();
    return { ok: true, heightCm };
  } catch {
    return { ok: false, message: 'Tide report unavailable' };
  }
}

const report = reportTide(async () => 0);
report.then(result => {
  if (!result.ok || result.heightCm !== 0) throw new Error('Bad report');
});
```

Remember. Await where you can act on rejection and use a distinct state for failure.

Try the encounter

LEVEL 06 / FORGE

The encounter

Checkpoint

If the loader resolves with 0, which result should reportTide fulfill with?

A. { ok: false, message: 'Tide report unavailable' }

B. { ok: true, heightCm: 0 }

C. undefined

D. A rejected promise

Code challenge

Implement readGate. Await the injected status function, return open or closed on success, and return offline if the function throws or rejects.

Level 6 starter

```
type Gate = 'open' | 'closed' | 'offline';

async function readGate(readOpen: () => Promise<boolean>): Promise<Gate> {
  throw new Error('Implement readGate');
}
```

Read the debrief · Back to the field manual

LEVEL 06 / EXPLAIN

Debrief and side quest

Checkpoint. B. A fulfilled promise can carry zero. The catch branch applies to a thrown error or rejected promise, regardless of the fulfillment values.

Hint one

Await the callback inside the try block so synchronous throws and rejections share the same failure branch.

Hint two

Map a resolved `false` to `closed`. Use `offline` only inside `catch`.

Compare your solution

A gate may be reachable and closed. Treating false as a request failure would conflate operational state with connectivity. Test resolved true, resolved false, and rejection.

Side quest

Preserve a safe diagnostic string on the offline result without assuming every rejected value is an Error.

Continue to Level 07 · World map

LEVEL 07 / FIELD MANUAL

The Dispatch Desk

Assemble validation and routing into a small public module with a clear interface.

A module can keep local helpers private and export the pieces other code should use. In this dispatch module the caller sees the decision contract and one routing function. The normalization step stays private because the caller does not need to depend on the exact trimming and uppercasing procedure.

A regular import can name a value the JavaScript runtime needs, such as a function. `import type` names a TypeScript declaration used only during checking and disappears from the emitted JavaScript. When you split the example into files, use the module settings and file specifiers required by your runtime. In a Node ESM setup that runs emitted JavaScript, a relative import of the compiled module can use `./dispatch.js` in the TypeScript source.

Keep routing separate from input and output. A pure decision function lets tests pass it a destination without opening a terminal or contacting a dock. A separate adapter can read the input, invoke the function, and show the result. Keep the exported return type explicit so a refactor cannot quietly change the public outcomes.

Worked example / Level 07

Listing 7.1 TypeScript example

```
export type Decision =
  | { kind: 'route'; dock: number }
  | { kind: 'invalid'; reason: string };

function normalizeDestination(value: string): string {
  return value.trim().toUpperCase();
}

export function decide(destination: string): Decision {
  switch (normalizeDestination(destination)) {
    case 'NORTH': return { kind: 'route', dock: 1 };
    case 'SOUTH': return { kind: 'route', dock: 2 };
    default: return { kind: 'invalid', reason: 'Unknown destination' };
  }
}

const decision = decide(' south ');
if (decision.kind !== 'route' || decision.dock !== 2) {
  throw new Error('Wrong dock');
}
```

Remember. Export the contract a caller needs and test the decision logic independently of its input adapter.

Try the encounter

LEVEL 07 / FORGE

The encounter

Checkpoint

Which declaration should a caller use when it only needs the Decision type for checking?

A. import type { Decision } from './dispatch.js'

B. const Decision = JSON.parse(...)

C. new Decision()

D. await Decision

Code challenge

Implement a module function that routes north to dock 1 and south to dock 2 after normalizing case and whitespace. Report invalid for other values. Export the decision type and the function.

Level 7 starter

```
export type Decision =
  | { kind: 'route'; dock: number }
  | { kind: 'invalid'; reason: string };

export function decide(destination: string): Decision {
  throw new Error('Implement decide');
}
```

Read the debrief · Back to the field manual

LEVEL 07 / EXPLAIN

Debrief and side quest

Checkpoint. A. `import type` gives the checker access to the declaration and emits no runtime import for it.

Hint one

Normalize once using `trim()` and `toUpperCase()`.

Hint two

Test `north`, `SOUTH` , and a destination with no matching branch.

Compare your solution

A caller can switch on the tagged decision without parsing a message or catching an exception for an ordinary invalid destination. The standalone module can be tested with simple inputs and no infrastructure.

Side quest

Build a separate presentation module that imports the type and formats each decision. Keep text formatting out of the routing module.

Continue to Level 08 · World map

LEVEL 08 / FIELD MANUAL

Keys That Carry Their Values

Use a generic key constraint and indexed access to preserve the relationship between a property name and its value.

A helper that reads one property should return the type of that specific property. `keyof T` names the keys of the object type, while `T[K]` is the type of a property at key `K`. A generic parameter `K extends keyof T` ties a caller's key to the object, so a typo fails checking and a precise key preserves a precise result.

Compare two different outputs when reading a `string` ID and a `number` capacity. A return type written as `T[keyof T]` would allow any property value type, so both calls would return a union. Capturing the selected key as its own parameter retains information already present at the call site.

An externally supplied string still needs a runtime decision about which fields are permitted. When the key is a union of known keys, its value type is a corresponding union. The generic relationship expresses exactly what was known at the call site and cannot make an unknown key safe by itself.

Worked example / Level 08

Listing 8.1 TypeScript example

```
function field<T, K extends keyof T>(record: T, key: K): T[K] {
  return record[key];
}

const dock = { id: 'N-1', capacity: 24 };
const id: string = field(dock, 'id');
const capacity: number = field(dock, 'capacity');
if (id !== 'N-1' || capacity !== 24) throw new Error('Wrong value');
```

Remember. Capture the caller's selected key as a type parameter when the result depends on that key.

Try the encounter

LEVEL 08 / FORGE

The encounter

Checkpoint

What type should `field(dock, 'capacity')` infer for the shown dock object?

A. keyof typeof dock

B. string | number

C. unknown

D. number

Code challenge

Implement a helper that gets a property from each object in a read-only list. Keep the output type linked to the selected property, so choosing a numeric field gives number[].

Level 8 starter

```
function pluck<T, K extends keyof T>(
  rows: readonly T[], key: K
): T[K][] {
  throw new Error('Implement pluck');
}
```

Read the debrief · Back to the field manual

LEVEL 08 / EXPLAIN

Debrief and side quest

Checkpoint. D. The literal key `capacity` determines `K`. Indexing the inferred dock type with that key gives `number`.

Hint one

The key constraint establishes that `row[key]` can be read for every row of type T.

Hint two

Return `rows.map(row => row[key])` and assign the result to a variable typed as the expected property array.

Compare your solution

The same code can return string[] or number[] because the return type keeps the key relationship. Test an empty list and a list of several objects with both key kinds.

Side quest

Accept an allowlisted field name from a text input and explain where the runtime narrowing occurs before calling pluck.

Continue to Level 09 · World map

LEVEL 09 / FIELD MANUAL

The Registry Blueprint

Derive related record shapes and use satisfies to check configuration without assigning a wider type to it.

An operations record and its editable patch often share fields. A mapped type can copy each allowed field while applying a modifier. The example excludes `id`, which remains stable, and makes every remaining field optional. A new field in the original record automatically becomes available to the patch according to this policy.

An absent patch property means leave the value alone. This example's runtime also preserves the previous value for explicit `undefined`. Enable `exactOptionalPropertyTypes` separately from `strict` if callers should be required to omit an optional property whose declared value type excludes `undefined`. Check untrusted patch values at the boundary.

Use `satisfies` when an expression needs to conform to a shape and you also want to work with the expression's own inferred type. The registry example checks that each required dock has a valid entry. `as const` preserves the literal capacity values and gives a read-only view in the type system. Neither construct freezes the runtime object.

Worked example / Level 09

Listing 9.1 TypeScript example

```
type Dock = { id: string; capacity: number; open: boolean };
type EditableKey = Exclude<keyof Dock, 'id'>;
type DockPatch = { [K in EditableKey]?: Dock[K] };

const defaultDocks = {
  north: { capacity: 24 },
  south: { capacity: 12 },
} as const satisfies Record<'north' | 'south', { capacity: number }>;

function applyPatch(dock: Dock, patch: DockPatch): Dock {
  return {
    id: dock.id,
    capacity: patch.capacity ?? dock.capacity,
    open: patch.open ?? dock.open,
  };
}

const cap: 24 = defaultDocks.north.capacity;
const next = applyPatch({ id: 'N', capacity: cap, open: true }, { open: false });
if (next.open) throw new Error('Dock was not closed');
```

Remember. Derive shape changes from the source contract and spell out what omission means.

Try the encounter

LEVEL 09 / FORGE

The encounter

Checkpoint

What does the `satisfies Record<...>` check on defaultDocks?

A. The registry has the required keys and compatible capacity values

B. The object is frozen when JavaScript runs

C. Every capacity is a positive integer

D. All later reads return the same capacity after mutation

Code challenge

Define a patch type derived from Dock that allows changes to capacity and open only. Apply an omitted patch field by preserving the previous value. Preserve false and zero when explicitly provided.

Level 9 starter

```
type Dock = { id: string; capacity: number; open: boolean };
type DockPatch = never; // Replace with a type derived from Dock.

function applyPatch(dock: Dock, patch: DockPatch): Dock {
  throw new Error('Implement applyPatch');
}
```

Read the debrief · Back to the field manual

LEVEL 09 / EXPLAIN

Debrief and side quest

Checkpoint. A. The checker validates the expression against the required shape. The literal types come from `as const`, and runtime values still follow JavaScript rules.

Hint one

`Partial<Omit<Dock, 'id'>>` expresses the same shape as the mapped type in the example.

Hint two

Read only the fields you intend to update. Use `??` so false and zero survive.

Compare your solution

A type-level patch narrows the API for typed callers. Reading the specific editable fields at runtime also protects the identifier if an object with extra properties reaches this function through structural compatibility.

Side quest

Add a removable note and decide how to represent preserve, replace, and clear as three different operations.

Continue to Level 10 · World map

LEVEL 10 / FIELD MANUAL

States That Refuse to Drift

Build a finite state transition and make a missing case fail TypeScript checking with never.

Operations gain states as they move through the harbor. A queued job can start, a running job can complete, and a completed job stays completed. Store the tag and any state-specific fields together so a job can never be both completed and waiting in the typed state model.

A switch can narrow the state across every branch. At the end, a function accepting `never` makes the expectation of complete coverage visible to the checker. Add a fourth state and the old switch will produce a type error at the call to that function until the new case is handled.

Exhaustive handling checks the listed variants. Whether a transition is legal depends on the domain. Keep the transition pure while you are designing it, then let a caller decide when to persist the new state or perform an effect. Validate input at the boundary before the transition receives it.

Worked example / Level 10

Listing 10.1 TypeScript example

```
type Job =
  | { state: 'queued'; id: string }
  | { state: 'running'; id: string }
  | { state: 'completed'; id: string };

function unreachable(value: never): never {
  throw new Error(`Unhandled state: ${String(value)}`);
}

function advance(job: Job): Job {
  switch (job.state) {
    case 'queued': return { state: 'running', id: job.id };
    case 'running': return { state: 'completed', id: job.id };
    case 'completed': return job;
    default: return unreachable(job);
  }
}

if (advance({ state: 'queued', id: 'J-2' }).state !== 'running') {
  throw new Error('Wrong transition');
}
```

Remember. Pair a tagged state model with an exhaustive switch and explicit transition rules.

Try the encounter

LEVEL 10 / FORGE

The encounter

Checkpoint

What happens if Job gains a cancelled variant and advance stays as written?

A. Cancelled automatically falls into completed

B. The never check fails because cancelled remains unhandled

C. The compiler silently adds a cancelled branch

D. Queued jobs can no longer be constructed

Code challenge

Add a cancelled state with a reason. Update advance so cancelled and completed jobs are terminal, queued jobs become running, and running jobs complete. Keep the exhaustive check.

Level 10 starter

```
type Job =
  | { state: 'queued'; id: string }
  | { state: 'running'; id: string }
  | { state: 'completed'; id: string };

function unreachable(value: never): never {
  throw new Error(`Unhandled state: ${String(value)}`);
}

function advance(job: Job): Job {
  throw new Error('Implement advance');
}
```

Read the debrief · Back to the field manual

LEVEL 10 / EXPLAIN

Debrief and side quest

Checkpoint. B. A cancelled job would reach the default branch, so the argument could not be assigned to the parameter typed as never.

Hint one

Extend the union with `{ state: 'cancelled'; id: string; reason: string }`.

Hint two

Return the same job object in terminal branches so the cancellation reason is retained.

Compare your solution

Each state has a clear successor. The compiler checks that every declared variant is handled, while tests can assert that terminal states keep their original data.

Side quest

Represent a retry event and return a typed rejection if someone tries to retry a completed job.

Continue to Level 11 · World map

LEVEL 11 / FIELD MANUAL

The Typed Message Board

Keep a message's tag correlated with the payload expected by its handler.

The message board carries different payloads for arrivals and departures. Keep the payload attached to its literal tag in one discriminated object. A handler switch then narrows the pair together. If you split the tag and payload into two independent union-typed variables, a caller could pair the wrong payload with a valid tag.

A payload map gives every event name a contract. A mapped type can derive one tagged object per key, and indexing the resulting object type with the whole key union produces a union of those cases. The example uses this to keep the event declaration in one place.

The dispatcher still needs a branch for each event it handles. A table of callbacks can also express event-specific parameter types, but calling a handler selected by a union key needs evidence that its argument corresponds to that key. Keep the association explicit at the call site and use an exhaustive switch when it is the clearest way to show the correlation.

Worked example / Level 11

Listing 11.1 TypeScript example

```
type Payloads = {
  arrive: { dock: number };
  depart: { vessel: string };
};
type Notice = {
  [K in keyof Payloads]: { kind: K; payload: Payloads[K] }
}[keyof Payloads];

function announce(notice: Notice): string {
  switch (notice.kind) {
    case 'arrive': return `Dock ${notice.payload.dock} occupied`;
    case 'depart': return `${notice.payload.vessel} departed`;
    default: {
      const missed: never = notice;
      throw new Error(`Unhandled notice: ${String(missed)}`);
    }
  }
}

const text = announce({ kind: 'depart', payload: { vessel: 'Gull' } });
if (text !== 'Gull departed') throw new Error(text);
```

Remember. Construct a union of valid tag and payload pairs and preserve the pair through dispatch.

Try the encounter

LEVEL 11 / FORGE

The encounter

Checkpoint

What is the benefit of deriving Notice as a union from Payloads?

A. It automatically validates external JSON messages

B. It guarantees the runtime delivery order

C. It ties each literal event tag to that event's own payload

D. It makes every payload optional

Code challenge

Add a delay notice with a vessel and a number of minutes. Produce `Gull delayed 5 min` for that case and keep the switch exhaustive.

Level 11 starter

```
type Payloads = {
  arrive: { dock: number };
  depart: { vessel: string };
};
type Notice = {
  [K in keyof Payloads]: { kind: K; payload: Payloads[K] }
}[keyof Payloads];

function announce(notice: Notice): string {
  throw new Error('Implement announce');
}
```

Read the debrief · Back to the field manual

LEVEL 11 / EXPLAIN

Debrief and side quest

Checkpoint. C. The mapped type builds each allowed pairing separately. The union does not admit a depart tag with an arrive payload.

Hint one

Add the event and payload in the map. The Notice union will pick up the new key automatically.

Hint two

Add a delay branch and read both fields from `notice.payload` after narrowing the tag.

Compare your solution

The type map is the source of the payload contract. The switch ties the runtime behavior to that contract, and its default never assignment detects a forgotten branch when a message type is added.

Side quest

Define a handler-table type from Payloads that requires one correctly typed callback for each event name.

Continue to Level 12 · World map

LEVEL 12 / FIELD MANUAL

Dispatching Several Calls

Represent the outcome of each asynchronous request while keeping the relationship to the original input order.

The scheduler can ask several docks for a status independently. Starting each request before awaiting them allows overlap. A sequential loop with an await before launching the next request is appropriate when an operation depends on the previous result or when the service requires serial access.

`Promise.all` returns fulfillment values in input order if every input fulfills. It rejects if any input rejects, and the remaining work may still continue. `Promise.allSettled` lets you inspect every outcome. Another approach is to catch rejection inside each operation, convert it to an explicit failure result, and gather those fulfilled result objects with `Promise.all`.

The example preserves the dock identifier in each result. That makes a failure actionable after results are filtered or logged separately. This example starts one operation for every input. A production scheduler with an unbounded list should add an explicit limit on active requests and decide whether it supports cancellation.

Worked example / Level 12

Listing 12.1 TypeScript example

```
type Probe =
  | { dock: string; ok: true; open: boolean }
  | { dock: string; ok: false; reason: string };

async function probeOne(
  dock: string, query: (dock: string) => Promise<boolean>,
): Promise<Probe> {
  try {
    return { dock, ok: true, open: await query(dock) };
  } catch {
    return { dock, ok: false, reason: 'Unavailable' };
  }
}

function probeAll(
  docks: readonly string[], query: (dock: string) => Promise<boolean>,
): Promise<Probe[]> {
  return Promise.all(docks.map(dock => probeOne(dock, query)));
}
```

Remember. Choose a collection operation whose error behavior matches what the caller needs to retain.

Try the encounter

LEVEL 12 / FORGE

The encounter

Checkpoint

If the second dock answers before the first and both finish, which order does probeAll return?

A. Whichever order TypeScript infers

B. Completion order

C. Sorted by dock name

D. The same order as docks

Code challenge

Implement probeAll using an injected query. Preserve the order and name of every dock, returning a failure result for just the docks whose queries throw or reject.

Level 12 starter

```
type Probe =
  | { dock: string; ok: true; open: boolean }
  | { dock: string; ok: false; reason: string };

async function probeAll(
  docks: readonly string[], query: (dock: string) => Promise<boolean>,
): Promise<Probe[]> {
  throw new Error('Implement probeAll');
}
```

Read the debrief · Back to the field manual

LEVEL 12 / EXPLAIN

Debrief and side quest

Checkpoint. D. `Promise.all` preserves input order in its output array, even when individual operations finish in a different order.

Hint one

Use an async helper or typed async map callback so each operation catches its own failure.

Hint two

Pass the resulting promises to `Promise.all`. Return a Probe in both branches of the per-dock operation.

Compare your solution

A failed request no longer discards successes from other docks. A result carries its identity, and the ordered array can still be paired with the caller's inputs. Verify a mix of true, false, and a rejection.

Side quest

Sketch how you would enforce at most four active queries. Identify where an active slot is released on failure.

Continue to Level 13 · World map

LEVEL 13 / FIELD MANUAL

Stamping an Identifier

Validate a string once at the boundary and give the checked identifier a distinct compile-time type.

Dock IDs and shipment IDs may both be strings. A function accepting either as a plain string cannot catch callers that mix up the two. A brand creates a type-level distinction using an intersection with a unique marker. A checked constructor is then the normal route from an untrusted string to the stronger type.

The parser in this example checks the entire identifier before asserting the brand. The assertion is small and sits directly after the runtime evidence. At runtime the value remains a string. Serializing it sends the text, and a receiving service should validate that text against its own boundary contract.

A format check says that an ID looks valid. It cannot prove that a corresponding shipment exists, belongs to a caller, or can be modified. Perform those checks in the service that owns the records. A useful brand name reflects the specific guarantee the parser establishes.

Worked example / Level 13

Listing 13.1 TypeScript example

```
declare const shipmentMark: unique symbol;
type ShipmentId = string & { readonly [shipmentMark]: true };

function parseShipmentId(value: unknown): ShipmentId | undefined {
  if (typeof value !== 'string' || !/^S-[0-9]{4}$/.test(value)) {
    return undefined;
  }
  return value as ShipmentId;
}

function labelFor(id: ShipmentId): string {
  return `Shipment ${id}`;
}

const id = parseShipmentId('S-0042');
if (!id || labelFor(id) !== 'Shipment S-0042') {
  throw new Error('Invalid identifier');
}
```

Remember. Keep the brand assertion beside the runtime check that establishes its limited guarantee.

Try the encounter

LEVEL 13 / FORGE

The encounter

Checkpoint

What guarantee does a value from parseShipmentId carry?

A. Its format is S- followed by four ASCII digits

B. The shipment exists

C. The current caller owns the shipment

D. The JavaScript value carries a runtime symbol property

Code challenge

Make a distinct DockId brand and a parser that accepts `D-` followed by exactly two ASCII digits. The value should be usable as a string after checking.

Level 13 starter

```
declare const dockMark: unique symbol;
type DockId = string & { readonly [dockMark]: true };

function parseDockId(value: unknown): DockId | undefined {
  throw new Error('Implement parseDockId');
}
```

Read the debrief · Back to the field manual

LEVEL 13 / EXPLAIN

Debrief and side quest

Checkpoint. A. The regular expression validates the format. The brand is compile-time information, and database existence and authorization require separate evidence.

Hint one

Start by checking `typeof value === 'string'` and then match `^D-[0-9]{2}$`.

Hint two

Return the branded assertion only after a successful match. Test `D-00`, `S-00`, `D-123`, and a number.

Compare your solution

The format check accepts zero digits inside an otherwise valid ID and rejects a shipment prefix. The separate marker lets functions enforce which category of checked ID they expect.

Side quest

Add a lookup that can return missing for a valid-looking dock ID. Explain why that check belongs after parsing.

Continue to Level 14 · World map

LEVEL 14 / FIELD MANUAL

Restore the Harbor Dispatch

Connect a runtime decoder to a typed decision and a small duplicate guard with an explicit boundary.

A transport delivers the same dispatch instruction twice. A command ID gives the dispatch desk a way to recognize a repeat. Start by validating a parsed value before creating a command. Give the command a literal destination union so the rest of the application works with a short list of valid destinations.

A `Map` can remember which destination an ID was assigned to. A repeated command with the same destination gets a duplicate response, and reusing an ID for another destination gets a conflict. The policy matters because treating a different command as a harmless repeat could hide an upstream bug. Return tagged decisions so callers can report each case distinctly.

This example makes a decision and records it synchronously in a single in-memory map. Its duplicate guarantee ends when that map is discarded. A real service needs durable state, a defined retention period, and a plan to coordinate deduplication with the side effect. Do not infer exactly-once delivery from a type annotation or the presence of an ID.

Worked example / Level 14

Listing 14.1 TypeScript example

```
type Destination = 'north' | 'south';
type Command = { id: string; to: Destination };
type Decision =
  | { kind: 'route'; dock: number }
  | { kind: 'duplicate' }
  | { kind: 'conflict' };

function decode(value: unknown): Command | undefined {
  if (typeof value !== 'object' || value === null) return undefined;
  if (Array.isArray(value)) return undefined;
  if (!Object.hasOwn(value, 'id') || !('id' in value)) return undefined;
  if (!Object.hasOwn(value, 'to') || !('to' in value)) return undefined;
  if (typeof value.id !== 'string' || value.id.trim() === '') {
    return undefined;
  }
  if (value.to !== 'north' && value.to !== 'south') return undefined;
  return { id: value.id.trim(), to: value.to };
}

function dispatch(command: Command, seen: Map<string, Destination>): Decision {
  const previous = seen.get(command.id);
  if (previous === command.to) return { kind: 'duplicate' };
  if (previous !== undefined) return { kind: 'conflict' };
  seen.set(command.id, command.to);
  return { kind: 'route', dock: command.to === 'north' ? 1 : 2 };
}
```

Remember. Validate at the boundary, keep decisions explicit, and state the lifetime of every operational guarantee.

Try the encounter

LEVEL 14 / FORGE

The encounter

Checkpoint

What should dispatch return if an ID used for north arrives again with destination south?

A. Route it to dock 2 as a new ID

B. Return a conflict

C. Erase the original entry

D. Treat it as an identical duplicate

Code challenge

Implement handle with a supplied Map. A command is a non-array record with its own nonblank string id and to of 'north' or 'south'. Trim the id. Return invalid for malformed commands, route for a fresh ID, duplicate for a matching repeat, and conflict if a repeat changes the destination. Only a fresh route updates the map.

Level 14 starter

```
type Destination = 'north' | 'south';
type Decision =
  | { kind: 'route'; dock: number }
  | { kind: 'duplicate' }
  | { kind: 'conflict' }
  | { kind: 'invalid' };

function handle(value: unknown, seen: Map<string, Destination>): Decision {
  throw new Error('Implement handle');
}
```

Read the debrief · Back to the field manual

LEVEL 14 / EXPLAIN

Debrief and side quest

Checkpoint. B. The same ID now refers to different work. Reporting a conflict preserves the first decision and makes the mismatched repeat visible.

Hint one

Narrow the input before looking it up. Extract the trimmed ID and destination only after checking both.

Hint two

Compare a saved destination with the incoming destination. Add the ID only on the fresh route path.

Compare your solution

Test one valid route, an identical repeat, a repeat for another destination, and an invalid object. Check the map after each call. These cases distinguish validation, conflict detection, and the one path that changes state.

Side quest

Sketch a restart-safe version. Define what is persisted, how long command IDs are retained, and what outcome a caller should see if delivery fails after dispatch recorded the ID.

Revisit the world map · Return to Level 01